Lista de Exercícios 8 - Revisão

1. Considere duas estruturas de dados do tipo pilha, denominadas A e B, com as sequências de valores (11, 33, 22) e (44, 22, 11) respectivamente, em que o elemento mais à direita de cada sequência refere-se ao topo da pilha. Sejam as operações: (i) desempilha(P), que remove um elemento da pilha P e retorna esse elemento; (ii) empilha(P, E), que insere o elemento E na pilha P; e (iii) topo(P), que retorna o elemento que está no topo da pilha P. Após executar a expressão "empilha(B, desempilha(A) + desempilha(A) - desempilha(B) + topo(B))", a sequência de elementos de B será:

a) (44, 22, 33).

b) (44, 22, 44).

c) (44, 22, 55).

d) (44, 22, 66).

e) (44, 22, 88).

2. Considerando uma fila dinâmica, definida pela *struct* apresentada abaixo, complete as lacunas na função *ordena*, a qual deve realizar a ordenação dos elementos da fila em ordem crescente.

```
struct NoFila
{
    int dado;
    NoFila *prox;
};

struct Fila
{
    NoFila *inicio;
    NoFila *fim;

    Fila(){
        inicio = NULL;
        fim = NULL;
    }
};
void ordena(Fila *F1)
{
    bool trocou;
    do{
        trocou = false;
        NoFila *no = F1->inicio;
        while (no!=NULL && ________________ !=NULL)
        {
            if(no->dado > ____________________)
            {
                 int aux = no->dado;
                no->dado = ________________;
                ________________ = aux;
                trocou = true;
            }
            no = no->prox;
        }

    }while(trocou==true);
}
```

3. O código apresentado abaixo contém uma pilha e uma fila, sendo que as duas estruturas de dados são dinâmicas e realizam operações de inserção e retirada. Especifique os valores da pilha que serão exibidos da linha 33 do código, indicando o topo e a base da pilha.

```
 7  int main(void)
 8  {
 9      Fila f;
10      Pilha p;
11      enfileiraF(&f, 5);
12      enfileiraF(&f, 2);
13      enfileiraF(&f, -3);
14      enfileiraF(&f, 9);
15      enfileiraF(&f, 1);
16      enfileiraF(&f, 0);
17      empilhaP(&p, 4);
18      empilhaP(&p, 1);
19      empilhaP(&p, 2);
20      empilhaP(&p, 8);
21      empilhaP(&p, 0);
22      empilhaP(&p, -1);
23      empilhaP(&p, 2);
24
25      empilhaP(&p, desenfileiraF(&f) - desempilhaP(&p));
26      empilhaP(&p, desempilhaP(&p) - desempilhaP(&p));
27      enfileiraF(&f, desempilhaP(&p));
28      empilhaP(&p, desenfileiraF(&f) + desenfileiraF(&f));
29
30      while(!vaziaF(&f))
31          empilhaP(&p, desenfileiraF(&f) - p.topo->dado);
32
33      mostraP(&p);
34      return EXIT_SUCCESS;
35  }
```

4. Considere uma fila dinâmica, a *função* apresentada abaixo e as inserções na pilha especificadas na função principal (*main*). Especifique os elementos da pilha que serão impressos na linha 34, indicando o topo e a base da pilha.

```
 8  void funcao(Pilha *p)
 9  {
10      Pilha aux;
11      while(!vaziaP(p)){
12          if((p->topo !=NULL && p->topo->prox!=NULL)){
13              int soma = p->topo->dado + p->topo->prox->dado;
14              if(soma%2 == 0)
15                  empilhaP(&aux, p->topo->dado);
16          }
17
18          desempilhaP(p);
19      }
20      while(!vaziaP(&aux))
21          empilhaP(p, desempilhaP(&aux));
22  }
23
24  int main(void){
25      Pilha p;
26      empilhaP(&p, 5);
27      empilhaP(&p, 7);
28      empilhaP(&p, 3);
29      empilhaP(&p, 2);
30      empilhaP(&p, 1);
31      empilhaP(&p, 1);
32
33      funcao(&p);
34      mostraP(&p);
35      return EXIT_SUCCESS;
36  }//final do main
```

5. Considerando uma fila dinâmica, definida pela *struct* apresentada abaixo, complete as lacunas na função *furaFila*, a qual deve realizar a inserção no início da fila.

```
struct NoFila
{
    int dado;
    NoFila *prox;
};

struct Fila
{
    NoFila *inicio;
    NoFila *fim;

    Fila(){
        inicio = NULL;
        fim = NULL;
    }
};
void furaFila(Fila *f, int valor)
{
    NoFila *novo = new NoFila();
    if (novo == NULL)
        return;

    novo->dado = valor;
    novo->prox = ___________________________;
```

```
    if (f->inicio == ____________________________)
        f->fim = ____________________________;


    f->inicio = ____________________________;

}
```

6. Implemente uma função para deslocar *n* posições dos primeiros *k* elementos do início da fila**.**

O protótipo da função deve ser:
`void` **`deslocar`**`(Fila *f, int k, int n);`

O programa deve exibir na tela:

- Imprimir a fila antes da chamada da função `deslocar`.
- Imprimir a fila depois da chamada da função `deslocar`.

Observações:

- A fila armazena valores inteiros positivos.
- O deslocamento dos elementos somente deve ocorrer quando $n>0$ e $(k+n)$ <= quantidade de elementos da fila.

Exemplos de entrada e saída:

```
int main(void)
{
    int k, n;
    Fila f;
    enfileiraF(&f, 10);
    enfileiraF(&f, 1);
    enfileiraF(&f, 3);
    enfileiraF(&f, 5);
    enfileiraF(&f, 90);

    cout << "Teste 1\n";
    mostraF(&f);
    k=3; n=1;
    deslocar(&f, k, n);
    mostraF(&f);

    cout << "\nTeste 2\n";
    mostraF(&f);
    k=3; n=2;
    deslocar(&f, k, n);
    mostraF(&f);

    cout << "\nTeste 3\n";
    mostraF(&f);
    k=3; n=3;
    deslocar(&f, k, n);
    mostraF(&f);

    cout << "\nTeste 4\n";
    mostraF(&f);
    k=3; n=0;
    deslocar(&f, k, n);
    mostraF(&f);

    cout << "\nTeste 5\n";
```

```
Teste 1
início [10, 1, 3, 5, 90] fim
início [5, 10, 1, 3, 90] fim
```

```
Teste 2
início [5, 10, 1, 3, 90] fim
início [3, 90, 5, 10, 1] fim

Teste 3
início [3, 90, 5, 10, 1] fim
início [3, 90, 5, 10, 1] fim

Teste 4
início [3, 90, 5, 10, 1] fim
início [3, 90, 5, 10, 1] fim

Teste 5
início [3, 90, 5, 10, 1] fim
início [90, 5, 3, 10, 1] fim

Teste 6
início [90, 5, 3, 10, 1] fim
início [1, 90, 5, 3, 10] fim
```

```
    mostraF(&f);
    k=1; n=2;
    deslocar(&f, k, n);
    mostraF(&f);

    cout << "\nTeste 6\n";
    mostraF(&f);
    k=4; n=1;
    deslocar(&f, k, n);
    mostraF(&f);

    destroiF(&f);
    return EXIT_SUCCESS;

}
```

7. Implemente uma função para <u>encontrar</u> e <u>remover</u> o maior e o menor valor de uma pilha **P.**

O protótipo da função deve ser:

```
void removeMaiorMenor(Pilha *P)
```

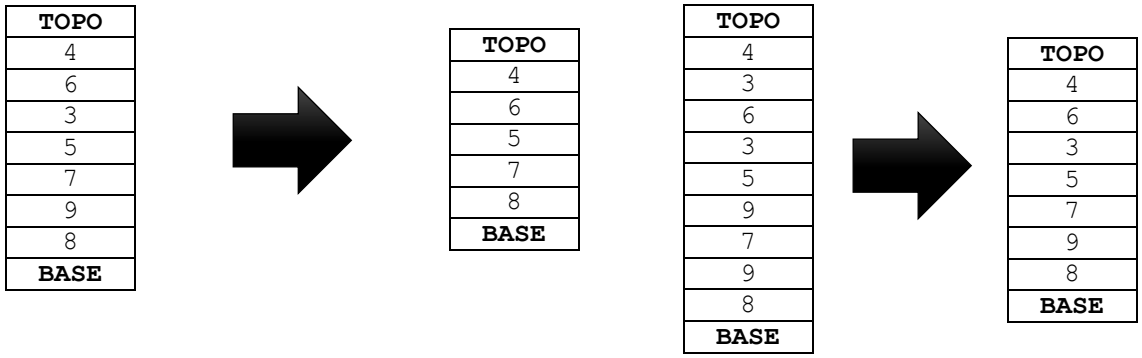
O programa deve exibir na tela:

- Imprimir a pilha **P** antes da chamada da função *removeMaiorMenor*.
- Imprimir a fila **P** depois da chamada da função *removeMaiorMenor*.

Observações:

- A pilha **P** é estática e armazenam no máximo 10 valores.
- A pilha **P** armazena valores inteiros positivos.
- A pilha **P** deve possuir no mínimo 2 valores armazenados.
- A função remove no <u>máximo 2 valores</u> da pilha **P**, ou seja, mesmo quando **P** armazena valores repetidos, a função removerá um valor para o maior e um valor para o menor.
- A remoção dos valores deve <u>manter a ordem relativa dos elementos</u> da pilha **P**, fazendo a remoção do topo para a base, conforme exemplos abaixo.

Exemplos:



8. Implemente uma função para encontrar os valores repetidos em uma pilha **P,** sendo que os valores encontrados devem ser armazenados em uma pilha **R**.

O protótipo da função deve ser:

```
void repetidos(Pilha *P, Pilha *R)
```

O programa deve exibir na tela:

- Imprimir a pilha **P** antes da chamada da função *repetidos.*
- Imprimir a pilha **R** depois da chamada da função *repetidos.*
- Imprimir a pilha **P** depois da chamada da função *repetidos.*

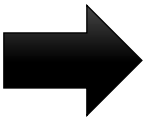
Observações:

- As pilhas **P** e **R** são estáticas e armazenam no máximo 15 valores.
- A pilha **R** está inicialmente vazia.
- A pilha **R** não pode conter elementos repetidos.
- A pilha **P** deve permanecer inalterada após a chamada da função *repetidos*.

Exemplo:

Pilha P

TOPO
1
4
3
8
4
0
7
6
7
5
7
9
8
BASE



Pilha R

TOPO
7
8
4
BASE

9. Considerando duas pilhas P1 e P2 e dois números inteiros **I** e **F**, implemente uma função para substituir um conjunto de valores de **P1** armazenados entre as posições **I** e **F** pelos elementos de **P2**.

O protótipo da função deve ser:

```
void substituir(Pilha *p1, Pilha *p2, int i, int f)
```

O programa deve exibir na tela:

- Imprimir as pilhas P1 e P2 antes da chamada da função `substituir`.
- Imprimir as pilhas P1 e P2 depois da chamada da função `substituir`.

Observações:

- Utilize a estrutura de dados Pilha dinâmica na implementação da questão.
- Não é permitido o uso da estrutura de dados Fila.
- As pilhas armazenam valores inteiros positivos.
- <u>Manter a ordem relativa dos elementos das pilhas.</u>
- A pilha P2 deve permanecer inalterada, ou seja, os elementos de P2 devem ser os mesmos e na mesma ordem, antes e depois da chamada da função *substituir*.
- Mostre uma mensagem de erro se o valor de **I** ou de **F** for inválido, com base nos seguintes casos:

- (**I**<0)
- (**F**<0)
- (**I**>**F**)
- (**I**>=**T**) ,sendo **T** o total de elementos de P1
- (**F**>=**T**) ,sendo **T** o total de elementos de P1

Exemplos de entrada e saída:

```
int main(void)
{
    setlocale(LC_ALL, "Portuguese");
    Pilha p1, p2;

    cout << "\n#Teste 1\n";
    empilhaP(&p1, 6);
    empilhaP(&p1, 5);
    empilhaP(&p1, 4);
    empilhaP(&p1, 3);
    empilhaP(&p1, 2);
    empilhaP(&p1, 1);

    empilhaP(&p2, 300);
    empilhaP(&p2, 200);
    empilhaP(&p2, 100);

    mostraP(&p1);
    mostraP(&p2);
    substituir(&p1, &p2, 2, 4);
    mostraP(&p1);
    mostraP(&p2);
    destroiP(&p1);
    destroiP(&p2);
```

```
#Teste 1
topo [1, 2, 3, 4, 5, 6] base
topo [100, 200, 300] base
topo [1, 2, 100, 200, 300, 6] base
topo [100, 200, 300] base
```

```
     cout << "\n#Teste 2\n";
    empilhaP(&p1, 6);
    empilhaP(&p1, 5);
    empilhaP(&p1, 4);
    empilhaP(&p1, 3);
    empilhaP(&p1, 2);
    empilhaP(&p1, 1);

    empilhaP(&p2, 300);
    empilhaP(&p2, 200);
    empilhaP(&p2, 100);

    mostraP(&p1);
    mostraP(&p2);
    substituir(&p1, &p2, 0, 1);
    mostraP(&p1);
    mostraP(&p2);
    destroiP(&p1);
    destroiP(&p2);
```

```
#Teste 2
topo [1, 2, 3, 4, 5, 6] base
topo [100, 200, 300] base
topo [100, 200, 300, 3, 4, 5, 6] base
topo [100, 200, 300] base

#Teste 3
topo [1, 2, 3, 4, 5, 6] base
topo [100, 200, 300] base
topo [100, 200, 300, 2, 3, 4, 5, 6] base
topo [100, 200, 300] base

#Teste 4
topo [1, 2, 3, 4, 5, 6] base
topo [100, 200, 300] base
topo [1, 100, 200, 300] base
topo [100, 200, 300] base
```

```
    cout << "\n#Teste 3\n";
    empilhaP(&p1, 6);
    empilhaP(&p1, 5);
    empilhaP(&p1, 4);
    empilhaP(&p1, 3);
    empilhaP(&p1, 2);
    empilhaP(&p1, 1);
```

```
    empilhaP(&p2, 300);
    empilhaP(&p2, 200);
    empilhaP(&p2, 100);

    mostraP(&p1);
    mostraP(&p2);
    substituir(&p1, &p2, 0, 0);
    mostraP(&p1);
    mostraP(&p2);
    destroiP(&p1);
    destroiP(&p2);


    cout << "\n#Teste 4\n";
    empilhaP(&p1, 6);
    empilhaP(&p1, 5);
    empilhaP(&p1, 4);
    empilhaP(&p1, 3);
    empilhaP(&p1, 2);
    empilhaP(&p1, 1);

    empilhaP(&p2, 300);
    empilhaP(&p2, 200);
    empilhaP(&p2, 100);

    mostraP(&p1);
    mostraP(&p2);
    substituir(&p1, &p2, 1, 5);
    mostraP(&p1);
    mostraP(&p2);
    destroiP(&p1);
    destroiP(&p2);
}
```

10. Implemente uma função que troque os elementos da primeira metade da Fila Dinâmica com os da segunda metade, mantendo a ordem relativa dentro de cada parte. A função deve atender aos seguintes requisitos:

a. A fila deve armazenar elementos do tipo Pessoa, contendo os campos **nome e salário**.

b. O protótipo da função deve ser:
`void` **`trocar`**`(Fila *f);`

c. <u>É permitido o uso de Filas auxiliares</u>. Outras estruturas de dados (como pilhas, listas, vetores, etc.) não são permitidas.

d. Imprimir a fila antes e depois da chamada da função trocar.

e. A impressão dos elementos da fila deve seguir o formato:

`Fila: inicio{[nome, salário], [nome, salário], ...}fim`

f. Se a fila tiver quantidade ímpar de elementos, o elemento central deve permanecer na mesma posição.

Um exemplo de entrada e saída são apresentados a seguir. O programa desenvolvido deve ser capaz de produzir a saída no <u>formato mostrado.</u>

```
include <iostream>
using namespace std;

struct Pessoa{
    string nome;
```

```
    float salario;
    Pessoa(){
        nome = "";
        salario = 0;
    }
    Pessoa(string n, float s){
        nome = n;
        salario = s;
    }
};

#include "fila-dinamica.hpp"

int main(void){
    setlocale(LC_ALL, "Portuguese");
    Fila fila;

    cout << "\n#Teste 1\n";
    enfileiraF(&fila, Pessoa("Fulano1", 1000));
    enfileiraF(&fila, Pessoa("Fulano2", 2000));
    enfileiraF(&fila, Pessoa("Fulano3", 3000));
    enfileiraF(&fila, Pessoa("Fulano4", 4000));
    enfileiraF(&fila, Pessoa("Fulano5", 5000));
    mostraF(&fila);
    trocar(&fila);
    mostraF(&fila);

    cout << "\n#Teste 2\n";
    destroiF(&fila);
    enfileiraF(&fila, Pessoa("Fulano1", 1000));
    enfileiraF(&fila, Pessoa("Fulano2", 2000));
    enfileiraF(&fila, Pessoa("Fulano3", 3000));
    enfileiraF(&fila, Pessoa("Fulano4", 4000));
    mostraF(&fila);
    trocar(&fila);
    mostraF(&fila);

    destroiF(&fila); //desalocar memória

    return EXIT_SUCCESS;
}
```

```
#Teste 1
Fila: inicio{[Fulano1, 1000], [Fulano2, 2000], [Fulano3, 3000], [Fulano4, 4000], [Fulano5, 5000]}fim
Fila: inicio{[Fulano4, 4000], [Fulano5, 5000], [Fulano3, 3000], [Fulano1, 1000], [Fulano2, 2000]}fim

#Teste 2
Fila: inicio{[Fulano1, 1000], [Fulano2, 2000], [Fulano3, 3000], [Fulano4, 4000]}fim
Fila: inicio{[Fulano3, 3000], [Fulano4, 4000], [Fulano1, 1000], [Fulano2, 2000]}fim
```